

# AI-POWERED CUSTOMER SUPPORT ASSISTANT

*Project Technical Report*

---

End-to-End System Design and Implementation

April 2026

**Srinidhi C R**

**Technology Stack:** GPT-4.1-mini · LangGraph ReAct · Qdrant · BM25 · CrossEncoder  
· LinearSVC TF-IDF · Gradio · SQLite · LangSmith

## Week 1 — Dataset Analysis, ML Modelling & Classifier

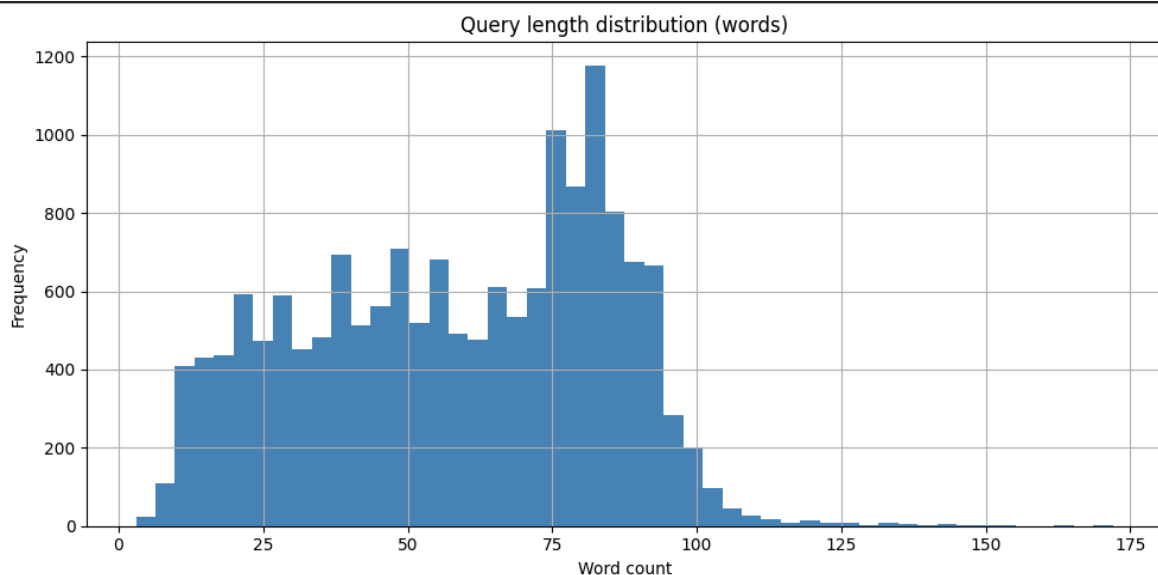
### 1.1 Dataset — Multilingual Customer Support Tickets

The source dataset is a multilingual collection of customer support tickets spanning English and German. Only the English portion was used. Each record contains a Subject line, a Body (the customer's email), a Queue (the raw support category), and up to eight tag fields. The dataset is highly imbalanced — the dominant class has roughly 20× more samples than the smallest class:

| Category (raw)                  | Sample Count |
|---------------------------------|--------------|
| Technical Support               | 4,737        |
| Product Support                 | 3073         |
| Customer Service                | 2410         |
| IT Support                      | 1942         |
| Billing and Payments            | 1595         |
| Returns and Exchanges           | 820          |
| Service Outages and Maintenance | 664          |
| Sales and Pre-Sales             | 513          |
| Human Resources                 | 348          |
| General Inquiry                 | 236          |

Query length analysis:

- Shortest query: 3 words. Longest: 173 words.
- Queries shorter than 15 words were removed.



## 1.2 Text Preprocessing

Because tickets are formatted as formal emails, raw text contains salutations, sign-offs, and boilerplate that add noise without discriminating signal. A cleaning pipeline was built in `text_cleaner.py`:

- Regex stripping of formal patterns: 'Dear Customer Support Team', 'Kind regards', 'I hope this message finds you well', etc.
- Template tag (<name>, <tel-num>) removal and lowercase normalisation.
- Tag normalisation: comma-separated tag values split, lowercased, deduplicated, and filtered to remove leakage keywords that directly reveal the category.
- Near-duplicate removal: TF-IDF cosine similarity > 0.75 triggers removal.

## 1.3 Models Evaluated

All models were evaluated under consistent conditions: 80/20 train-test split, 5-fold StratifiedKFold cross-validation, and Macro F1 as the primary metric (chosen because of class imbalance).

| Model                          | CV F1         | Test Accuracy | Macro F1      | Train Time   |
|--------------------------------|---------------|---------------|---------------|--------------|
| Logistic Regression (baseline) | 0.3766        | 0.5328        | 0.3987        | 27.78s       |
| Logistic Regression (weighted) | 0.5087        | 0.5273        | 0.5385        | 13.78s       |
| Random Forest (baseline)       | 0.5565        | 0.6599        | 0.6198        | 75.30s       |
| Random Forest (weighted)       | 0.5813        | 0.6650        | 0.6487        | 85.02s       |
| <b>Linear SVM (weighted)</b>   | <b>0.6232</b> | <b>0.6650</b> | <b>0.6696</b> | <b>3.32s</b> |
| Multinomial Naive Bayes        | 0.2196        | 0.4392        | 0.2312        | 1.13s        |
| XGBoost (weighted)             | —             | 0.5528        | 0.5535        | —            |
| Zero-Shot MiniLM               | —             | 0.2444        | 0.1914        | —            |

Linear SVM emerged as the clear winner: highest Macro F1 (0.67), fastest training at 3.32 seconds, and the most balanced precision-recall tradeoff.

## 1.4 Category Merging

A recurring pattern in the confusion matrices was that semantically similar categories were being confused with each other — IT Support / Technical Support / Service Outages all overlapped heavily in vocabulary, as did the customer-facing categories. Six fine-grained labels were merged into six canonical categories:

| Raw Queue Labels                                               | Canonical Category                |
|----------------------------------------------------------------|-----------------------------------|
| IT Support, Technical Support, Service Outages and Maintenance | Technical                         |
| Product Support, Customer Service, Sales and Pre-Sales         | Product Inquiry                   |
| Billing and Payments                                           | Billing and Payments (unchanged)  |
| Returns and Exchanges                                          | Returns and Exchanges (unchanged) |

| Raw Queue Labels | Canonical Category          |
|------------------|-----------------------------|
| Human Resources  | Human Resources (unchanged) |
| General Inquiry  | General Inquiry (unchanged) |

Results after merging:

| Model                          | Test Accuracy | Macro F1      |
|--------------------------------|---------------|---------------|
| Logistic Regression (weighted) | 0.6839        | 0.6405        |
| Random Forest (baseline)       | 0.7721        | 0.6286        |
| Random Forest (weighted)       | 0.7755        | 0.6527        |
| <b>Linear SVM (weighted)</b>   | <b>0.7966</b> | <b>0.7661</b> |
| Multinomial Naive Bayes        | 0.6352        | 0.3507        |
| XGBoost (baseline)             | 0.7029        | 0.5430        |
| XGBoost (weighted)             | 0.6885        | 0.6165        |
| XGBoost (focal + weighted)     | 0.7078        | 0.5455        |

After merging, Linear SVM improved from Macro F1 0.67 → 0.77.

## 1.5 Embedding-Based Features (MiniLM vs TF-IDF)

To check whether dense semantic representations could outperform TF-IDF, all-MiniLM-L6-v2 was used to encode queries into 384-dimensional vectors, then fed into the same classifiers:

| Model                          | Test Accuracy | Macro F1 |
|--------------------------------|---------------|----------|
| Embedding + LR (weighted)      | 0.43          | 0.3512   |
| Embedding + RF (weighted)      | 0.7571        | 0.5379   |
| Embedding + XGBoost (weighted) | 0.7764        | 0.7172   |
| Embedding + LinearSVM          | 0.3833        | 0.5121   |

TF-IDF outperforms embeddings across the board. The reason: TF-IDF captures sharp keyword distinctions (VPN, invoice dispute, network outage) that provide cleaner category separation.

## 1.6 Final Feature Engineering and Model Selection

After confirming TF-IDF superiority, the feature set was expanded in three steps:

- Tag integration: top-3 cleaned tags concatenated into the input string after leakage-keyword removal.
- Character n-grams: char\_wb (3-5) n-grams added via FeatureUnion to capture technical abbreviations and spelling variations.
- Hyperparameter tuning: C swept across values; C=5 found to be optimal.

The best performing configuration was a Pipeline combining a TF-IDF vectoriser (unigrams and bigrams, 50,000 features, sublinear TF, L2 normalisation) with a LinearSVC (C=2.0, class\_weight=balanced), trained on a combined string of **subject, cleaned body, and safe top-3 tags**. This matched the model code in train\_classifier.py exactly. SMOTE and SMOTETomek oversampling experiments were also run and produced comparable Macro F1 scores.

| Configuration                  | Test Accuracy | Macro F1      |
|--------------------------------|---------------|---------------|
| LinearSVM — multi input        | 0.7703        | 0.7373        |
| <b>LinearSVM — concat tags</b> | <b>0.8022</b> | <b>0.7796</b> |
| LinearSVM — char ngrams        | 0.6582        | 0.5877        |
| LinearSVM — word + char (C=5)  | 0.7999        | 0.7734        |
| LinearSVM — SMOTE Oversampling | 0.7874        | 0.7615        |
| LinearSVM — SMOTETomek         | 0.7841        | 0.7592        |

Refer to figure 1 for overall flow.

**Final Classifier:** LinearSVC + TF-IDF (word 1–2, 50k features, sublinear\_tf, L2) + safe top-3 tags + C=2.0, class\_weight=balanced → Macro F1: 0.7796 | Test Accuracy: 80.22% | Train time: 3.17s | Inference: ~0.09ms

## Week 2 — Full System Architecture and Implementation

### 2.1 Technology Stack

| Component           | Technology / Implementation                               |
|---------------------|-----------------------------------------------------------|
| LLM backbone        | OpenAI GPT-4.1-mini (generation, critique, query rewrite) |
| Embedding model     | sentence-transformers / all-MiniLM-L6-v2 (384-dim)        |
| Dense vector store  | Qdrant (self-hosted, cosine similarity)                   |
| Sparse index        | BM25Okapi via rank-bm25                                   |
| Reranker            | cross-encoder / ms-marco-MiniLM-L-6-v2                    |
| Ticket persistence  | SQLite via ticket_store.py                                |
| Agent orchestration | LangGraph ReAct agent                                     |
| UI                  | Gradio Blocks with custom CSS                             |
| Classifier          | Calibrated LinearSVC with TF-IDF (scikit-learn)           |
| Semantic cache      | Qdrant collection, cosine similarity threshold 0.80       |
| Observability       | LangSmith tracing (all agent + tool calls instrumented)   |

### 2.2 Knowledge Base Construction

#### 2.2.1 Dataset Preprocessing

The same cleaned dataset from Week 1 was used as the KB source. After filtering to english, merging categories, text cleaning, replacing PII placeholders (<name>, <tel\_num>) and deduplication, 9,555 unique rows remained:

| Category              | KB Entries |
|-----------------------|------------|
| Technical             | 4,366      |
| Product Inquiry       | 3,521      |
| Billing and Payments  | 874        |
| Returns and Exchanges | 471        |
| Human Resources       | 184        |
| General Inquiry       | 139        |
| Total                 | 9,555      |

#### 2.2.2 Chunk Construction

Each row is converted to a structured chunk embedding three fields rather than just the answer. This is the key design decision: many templated answers are nearly identical, so the query and subject provide discriminating context for dense retrieval.

**Chunk format:** Subject: <ticket subject> | Customer issue: <cleaned query> | Resolution: <cleaned answer>

### 2.2.3 Resolution Chunk Enrichment

Approximately 30–40% of base KB answers are acknowledgement-only responses. A generation script (`generate_resolution_kb.py`) was written to detect these using a pattern heuristic. For each flagged entry the script calls GPT-4.1-mini with a structured prompt instructing it to produce a 100-200 word resolution response containing a brief acknowledgement, three to five numbered troubleshooting steps specific to the issue, and an escalation path if the steps do not work. Up to 50 entries per category are processed, yielding approximately 200 additional resolution chunks.

### 2.2.4 Dense Index (Qdrant)

The dense index is built using `qdrant_store.py`. The all-MiniLM-L6-v2 sentence transformer encodes every chunk text into a 384-dimensional vector. The encoder is loaded from a local snapshot path (`transformer-models/`) if available, falling back to the HuggingFace hub only when the snapshot is missing. This makes retrieval reproducible in offline environments.

A Qdrant collection named `support_kb` is created with cosine distance. All chunk vectors and their full payloads are uploaded in batches of 500 using `upsert`. The category is stored in the payload so dense search can apply a category filter at query time, ensuring that retrieved chunks always match the classified category of the incoming query.

### 2.2.5 Sparse Index (BM25)

The sparse index is built using `bm25_store.py`. The full corpus of chunk texts is tokenised (lowercased, whitespace-split) and fed to BM25Okapi. The resulting index — containing IDF scores and document lengths — is serialised to a pickle file (`bm25_index.pkl`) and loaded lazily on the first search call. Like the dense search, BM25 filters results to the predicted category before returning the top-k documents.

## 2.3 Category Classifier

### 2.3.1 Inference

At inference time the `SupportClassifier.predict` method returns a dict with three fields: `category` (the predicted label), `confidence` (the max class probability from the calibrated model), and `latency_ms`. The confidence score is used throughout the pipeline to make routing decisions.

| Confidence Range  | Pipeline Behaviour                                             |
|-------------------|----------------------------------------------------------------|
| confidence < 0.25 | Route to clarify — query is too ambiguous to classify reliably |
| 0.25 to 0.65      | Retrieve and generate but apply critique carefully             |
| >= 0.65           | Full pipeline — high confidence in the category                |

## 2.4 Retrieval Pipeline

### 2.4.1 Query Rewriting

Before searching, the retrieve tool calls the LLM with the original query and the predicted category to produce a rewritten query that is more specific and domain-keyword-rich.

### 2.4.2 Hybrid Search

Dense search (Qdrant) is run with the rewritten query, filtered to the predicted category, returning up to 20 candidates. Sparse search (BM25) is run in parallel with the same rewritten query, also filtered and returning up to 20 candidates. The two candidate lists are then merged using Reciprocal Rank Fusion (RRF).

RRF assigns each document a score of  $1 / (k + \text{rank})$  for each list in which it appears ( $k=60$  is the standard smoothing constant). Documents that appear highly ranked in both lists accumulate higher combined scores and rise to the top of the fused list. This naturally rewards multi-signal agreement without requiring any calibration of score scales between the two retrievers.

### 2.4.3 Cross-Encoder Reranking

The fused candidate list is passed to the cross-encoder reranker (reranker.py). The model ms-marco-MiniLM-L-6-v2 scores every (query, chunk text) pair jointly, which allows it to capture fine-grained relevance signals. The top 5 documents by rerank score are returned. The rerank score is attached to each chunk and surfaced in the UI so support agents can judge relevance at a glance.

### 2.4.4 Semantic Cache

Before the retrieval pipeline fires, a semantic cache lookup is performed. The cache is a separate Qdrant collection (response\_cache) that stores vectors of previously processed queries alongside the full pipeline response they produced. If an incoming query's embedding is within cosine distance 0.80 of a cached query, the cached response is returned immediately, skipping classification, retrieval, generation, and critique entirely. This saves latency and LLM cost for repeat or near-repeat queries.

Cache entries are only written for substantive responses (longer than 20 words) and are skipped for replies that contain phrases like unable to find relevant information or please escalate, since those indicate the pipeline did not produce a useful answer.

### 2.4.5 Low-Relevance Fallback

After reranking, the chat handler inspects the best rerank score across the returned chunks. If no chunk exceeds a threshold of 1.5, the query is considered to have insufficient retrieval coverage. In that case the system returns a polite refusal message instead of generating a potentially hallucinated response, and asks the user to rephrase with more product-specific detail.



| Stage               | Description                                                                                              |
|---------------------|----------------------------------------------------------------------------------------------------------|
| Query Rewrite       | GPT-4.1-mini rewrites the user query to be more domain-specific (conditional — see Week 3 optimisations) |
| Dense Search        | Qdrant cosine search, category-filtered, top-20 candidates                                               |
| Sparse Search       | BM25 category-filtered, top-20 candidates (run in parallel with dense — see Week 3)                      |
| RRF Fusion          | Reciprocal Rank Fusion (k=60): $\text{score} = \sum 1/(k+\text{rank})$ per list                          |
| CrossEncoder Rerank | ms-marco-MiniLM-L-6-v2 scores (query, chunk) pairs — top 5 returned                                      |
| Rerank Score        | Attached to each chunk, shown in UI; <1.5 triggers low-relevance fallback                                |

| Method         | Retrieval Metrics                          |
|----------------|--------------------------------------------|
| Dense (Qdrant) | Hit@1: 86.3%   Hit@5: 97.5%   MRR@5: 0.909 |
| Sparse (BM25)  | Hit@1: 100%   Hit@5: 100%   MRR@5: 1.000   |
| Hybrid (RRF)   | Hit@1: 97.5%   Hit@5: 100%   MRR@5: 0.986  |

The perfect BM25 Hit@1 reflects the highly templated nature of the dataset — the original customer query text is directly embedded in the chunk, creating near-exact lexical overlap. Hybrid retrieval outperforms dense-only across all categories, confirming that the fusion strategy adds value even when BM25 is already strong.

## 2.5 LangGraph Agent Orchestration

A LangGraph ReAct agent (orchestrator.py) is given five tools and a strict decision policy via system prompt. The agent always calls tools in a defined order, never hallucinating facts not in retrieved documents.

| Tool     | Purpose                                                                                                              |
|----------|----------------------------------------------------------------------------------------------------------------------|
| classify | Always first. Calls the LinearSVC classifier. Predicts category + confidence.                                        |
| retrieve | Hybrid search: query rewrite → dense + sparse → RRF → CrossEncoder top-5.                                            |
| generate | Takes the query, category, and retrieved documents. Calls GPT-4.1-mini to produce a grounded draft response in JSON. |
| critique | LLM evaluates grounding, tone, completeness. Sets should_retry if score < 0.60.                                      |
| clarify  | Returns a clarifying question for out-of-scope / ambiguous / very low-confidence queries.                            |

## 2.6 Generate Tool

The generate tool builds a prompt that provides the support category, the original customer query, and the retrieved document texts. The LLM is instructed to extract every concrete fact from the context (system names, error types, known issues, policies, timelines),

acknowledge the customer's specific problem, include actionable steps if the context contains them, and suggest safe generic next steps when the context is only acknowledgement-style. It must never introduce facts, product names, or solutions not implied by the retrieved context.

## 2.7 Critique Tool

The critique tool sends the draft and the retrieved documents to the LLM for self-evaluation. It scores grounding (are all claims supported by the source documents?), tone (professional and empathetic?), and completeness (does it fully address the query?). If the overall score falls below the `critique_threshold` (0.60), the orchestrator is instructed to retry, retrieve and generate once. The critique score and hallucination flag are returned alongside the final result.

## 2.8 Full Query Flow (Refer figure 4)

Every user message passes through 19 sequential gates before a response is returned:

| Step | Check / Action             | What Happens                                                                                                       |
|------|----------------------------|--------------------------------------------------------------------------------------------------------------------|
| 1    | Empty message check        | Return immediately — no processing                                                                                 |
| 2    | Slash command intercept    | <code>commands.handle()</code> → <code>/help</code> <code>/new</code> <code>/resolve</code> <code>/status</code>   |
| 3    | Resolved ticket guard      | If ticket status = resolved → silently drop message                                                                |
| 4    | Active category lookup     | <code>get_ticket()</code> from SQLite → extract current category                                                   |
| 5    | Guardrails check           | Greeting / noise / topic-drift detection                                                                           |
| 6    | Low-information guard      | ≤3 chars or single vague word → reject before ticket creation                                                      |
| 7    | Call scheduling detection  | Pattern match → open scheduler panel                                                                               |
| 8    | Follow-up detection        | ≤12 words + trigger word + no domain KW → <code>answer_followup()</code>                                           |
| 9    | Turn-intent classification | <code>classify_turn_intent()</code> → <code>same_issue</code> / <code>new_issue</code> / <code>out_of_scope</code> |
| 10   | Ticket creation            | First message: <code>create_ticket()</code> → insert into SQLite                                                   |
| 11   | Semantic cache lookup      | Qdrant cosine check (threshold 0.80) → instant return on hit                                                       |
| 12   | Classify tool              | LinearSVC → predicted category + confidence                                                                        |
| 13   | Retrieve tool              | Query rewrite → Qdrant + BM25 → RRF → CrossEncoder top-5                                                           |
| 14   | Generate tool              | GPT-4.1-mini drafts response grounded in retrieved chunks                                                          |
| 15   | Critique tool              | LLM evaluates draft — retry once if score < 0.60 (see Week 3 conditional skip)                                     |
| 16   | Low-relevance fallback     | Best rerank score < 1.5 → refuse and ask for rephrasing                                                            |
| 17   | Cache store                | Write query+draft+chunks to <code>response_cache</code> if substantive                                             |
| 18   | SQLite persistence         | <code>add_message()</code> for user + assistant — <code>chunks_json</code> stored                                  |
| 19   | UI update                  | Chatbot history + chunk panel + ticket info + sidebar refresh                                                      |

### 2.8.1 Input Arrival and Slash Command Intercept

The Gradio UI calls the `chat()` generator function in `chat_handler.py`. The first check is whether the message starts with a forward slash. If it does, `commands.handle()` is called immediately, bypassing all downstream logic. The available slash commands are `/help`, `/new`, `/resolve`, and `/status`.

### 2.8.2 Active Ticket Context

If a ticket is already open, the active category is fetched from the SQLite tickets table. This category will be used throughout the remaining checks to determine whether the new message belongs to the existing ticket or represents something new.

### 2.8.3 General Guardrails

`guardrails.check()` is called next. It performs three checks in order:

- **Greeting detection.** If the message (lowercased) is in or overlaps with the GREETINGS set {hi, hello, hey, good morning, ...}, the system returns a welcome message without opening a ticket or touching the KB.
- **Noise / filler detection.** Single-word filler responses like ok, sure, thanks, got it are caught here and answered with a brief acknowledgement.
- **Topic drift detection.** If a ticket is open and the message contains keywords from a different support category with no overlap with the active category, the system warns the user to open a new ticket. This check only fires when the message is longer than five words.

### 2.8.4 Low-Information Guard

`guardrails.low_information_reply()` checks for accidental or underspecified inputs. A message is flagged as low-information if it is three characters or fewer with no domain signal, or if it is a single vague word with no domain signal. Inputs like hu, h, or test (with no KB-relevant content) are caught here, preventing accidental ticket creation.

### 2.8.5 Call Scheduling Detection

If the message contains any of the scheduling phrases (schedule a call, book a meeting, arrange a call, etc.), the chat handler opens the in-UI scheduling panel. The user is asked for an email address and a time slot. Once submitted, `schedule_call()` builds a confirmation email and attempts SMTP delivery if the environment variables `SMTP_HOST`, `SMTP_USER`, and `SMTP_PASSWORD` are configured. If SMTP is absent, the composed email draft is shown to the user in a preview box. The scheduling flow is entirely self-contained and does not interact with the KB pipeline.

### 2.8.6 Follow-Up Detection

`guardrails.is_followup()` checks whether the message is a conversational elaboration of the previous bot reply rather than a new standalone query. Four conditions must all be true for a message to be classified as a follow-up:

- A ticket is already open (`ticket_category` is not None).
- The message is short — twelve words or fewer.
- The message contains at least one of the follow-up trigger words: explain, elaborate, clarify, simpler, rephrase, what do you mean, more detail, break it down, etc.

- The message contains no domain keywords from any support category (so a message like explain my billing charge is correctly routed as a new query rather than a follow-up).

When a message is detected as a follow-up, `_handle_followup()` is called. This function fetches the last substantive assistant reply and its stored source chunks from SQLite via `get_last_bot_reply()`, `get_last_user_message()`, and `get_last_assistant_chunks()`. Only assistant messages that went through retrieval (i.e. those with `chunks_json` attached) are considered valid anchors — guardrail replies, routing redirects, and ticket banners are automatically skipped. The orchestrator then constructs a focused prompt anchored to the prior question, prior response, and prior source chunks, and invokes the LLM directly without running the full retrieve-generate-critique cycle. This keeps follow-up answers on-topic and avoids drifting into unrelated KB chunks.

| Helper                                   | Behaviour                                                                                    |
|------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>get_last_bot_reply()</code>        | Returns the most recent assistant message that has chunks attached — skips guardrail replies |
| <code>get_last_user_message()</code>     | Returns the user message that preceded the last substantive assistant reply                  |
| <code>get_last_assistant_chunks()</code> | Returns chunks from the most recent substantive assistant message                            |
| <code>get_prior_context()</code>         | Takes the last 3 substantive support exchange pairs, skipping noise turns                    |

*Why this works: `chunks_json` is the ground truth signal for a real support answer. Any assistant message without chunks is a guardrail reply, routing redirect, out-of-scope block, or ticket banner — none of which should anchor follow-up context. Filtering on chunks automatically skips noise regardless of what happened in between.*

## 2.8.7 Turn-Intent Classification

If none of the above short-circuit conditions triggered, `classify_turn_intent()` is called. This is the dedicated turn-routing layer that determines what category the message belongs to and whether it is consistent with the current ticket.

Internally it calls `SupportClassifier.predict()` and then `route_turn()`. The routing logic applies the following rules in priority order:

- If confidence < 0.35 OR the message matches a generic QA pattern (`who is`, `what is`, etc.) with no domain terms, the intent is set to `out_of_scope`. This `_looks_like_generic_qa()` check fires before the pipeline so off-topic questions never enter retrieval.
- If the predicted category differs from the active ticket category, confidence  $\geq 0.70$ , lexical overlap with the prior message is below 0.15, AND no ticket-domain keywords are present, the intent is set to `new_issue`. All three conditions must hold jointly to avoid false triggers on same-topic rephrasings.
- Otherwise the intent is `same_issue` and the pipeline continues.

### 2.8.8 Ticket Creation

If no ticket exists yet and the turn intent is `same_issue`, a new ticket is created. `make_ticket_id()` generates a UUID-based identifier (TKT-XXXXXXX). `create_ticket()` inserts a row into the SQLite tickets table with the ticket ID, predicted category, status open, creation timestamp, and a 120-character summary of the first message. A system message is injected into the chat history announcing the new ticket, its category, and the classifier confidence.

### 2.8.9 Pipeline Execution

The chat handler assembles the query by appending a compact conversation transcript from `get_prior_context()` (up to the last three substantive turns, 220 characters per message, skipping noise turns). The combined string is passed to `orchestrator.run()`, which executes the full `classify` → `retrieve` → `generate` → `critique` cycle described in Section 2.5.

### 2.8.10 Low-Relevance Check

After the pipeline completes, the chat handler checks the rerank scores of the returned chunks. If the best rerank score across all chunks is below 1.5 (no chunk crossed the relevance threshold), the draft is discarded and replaced with a polite message asking the user to rephrase with more specific detail. The chunks list is also cleared so no misleading source references appear in the UI.

### 2.8.11 Cache Store

If the final draft is substantive (more than 20 words and not a fallback refusal phrase), the query and draft are written to the semantic cache via `semantic_cache.store()`. The next sufficiently similar query will receive the cached response instantly.

### 2.8.12 Persistence and Response

Both the user message and the assistant draft are written to the SQLite messages table via `add_message()`. The chunks are serialised as JSON and stored in the `chunks_json` column so they can be recovered for follow-up grounding. The Gradio UI is updated with the new chat history, the refreshed ticket list, the ticket info header, and the source-chunk HTML panel.

## 2.9 SQLite Ticket and Message Store

All state is persisted in `data/tickets.db` via `ticket_store.py` — the single interface for all database operations. The LLM has no memory between calls; all conversational context is reconstructed from this database on each turn.

| Column     | Description                                 |
|------------|---------------------------------------------|
| id         | TKT-XXXXXXX — UUID-based ticket identifier  |
| category   | Predicted support category                  |
| status     | 'open' or 'resolved'                        |
| created_at | ISO timestamp                               |
| summary    | First 120 characters of the opening message |

| Column      | Description                                                             |
|-------------|-------------------------------------------------------------------------|
| id          | Auto-increment primary key                                              |
| ticket_id   | Foreign key to tickets.id                                               |
| role        | 'user' or 'assistant'                                                   |
| content     | Full message text                                                       |
| chunks_json | JSON-serialised source chunks (nullable) — used for follow-up grounding |
| created_at  | ISO timestamp                                                           |

## 2.10 User Interface

| Component          | Description                                                                 |
|--------------------|-----------------------------------------------------------------------------|
| Sidebar            | New Ticket button, ticket Radio list (SQLite-backed), Mark Resolved button  |
| Chat panel         | Gradio Chatbot with markdown rendering, ticket info header                  |
| Source chunk panel | HTML <details> block showing top-5 chunks with relevance scores and tags    |
| Scheduling panel   | Hidden until scheduling intent detected — email + slot + SMTP dispatch      |
| Composer row       | Cache toggle checkbox + message textbox + Send button (sticky bottom)       |
| Slash commands     | /help /new /resolve /status — bypass all pipeline logic                     |
| Resolved lock      | Mark Resolved disables composer; switching to resolved ticket also locks UI |

## Week 3 — Robustness Fixes, LangSmith Integration & Latency Optimisation

---

### 3.1 LangSmith Observability Integration

LangSmith tracing was integrated to provide full visibility into every pipeline run. Since the system uses LangChain and LangGraph, tracing is automatic once the environment variables are set — every agent reasoning step, tool call, LLM invocation, and token count is captured and visible in the LangSmith dashboard.

| Configuration     | Value                                                 |
|-------------------|-------------------------------------------------------|
| LANGSMITH_TRACING | true (set in .env)                                    |
| LANGSMITH_PROJECT | AI Powered Customer Support                           |
| LANGSMITH_API_KEY | ls___... (from LangSmith account)                     |
| Project URL       | app.smith.langchain.com → AI Powered Customer Support |

Each pipeline run creates one parent trace named `support_pipeline` with the full waterfall of child spans:

| Trace Span                      | Typical Latency            | Tokens |
|---------------------------------|----------------------------|--------|
| agent (initial reasoning)       | ~1.8s                      | 898    |
| tools → retrieve                | ~5.8s                      | 70     |
| agent (decide to call generate) | ~22s                       | 4.1K   |
| tools → generate                | ~11s                       | 1K     |
| agent (decide to call critique) | ~25s                       | 6.1K   |
| tools → critique                | ~1.2s                      | 737    |
| agent (final response)          | ~2.6s                      | 6.4K   |
| Total                           | ~70s (before optimisation) | 19.3K  |

The traces revealed that the dominant latency source is not the tools themselves but the LangGraph ReAct agent's own reasoning steps between tool calls — each inter-tool decision involves an LLM call over the full accumulated message thread, which grows with every tool result appended.

### 3.2 Latency Optimisation

LangSmith traces identified three concrete optimisation opportunities that could be implemented without replacing the ReAct agent architecture. All three were implemented:

### 3.2.1 Conditional Query Rewrite Skip

The retrieve tool previously always called GPT-4.1-mini to rewrite the query before searching, adding ~1.5s to every retrieval. Analysis showed that when the classifier is already confident (confidence  $\geq 0.75$ ), the original query is specific enough for good retrieval and the rewrite adds no value.

| Config Key              | Value | Effect                                                                                         |
|-------------------------|-------|------------------------------------------------------------------------------------------------|
| skip_rewrite_confidence | 0.75  | If classifier confidence $\geq 0.75$ , use original query directly — skip the LLM rewrite call |

The retrieve tool now accepts the confidence score as an input parameter passed from the classify tool. The system prompt was updated to instruct the agent to always pass confidence through to retrieve(). Estimated saving: ~1.5s on high-confidence queries.

### 3.2.2 Parallel Dense and Sparse Search

Dense search (Qdrant, network I/O) and sparse search (BM25, in-memory) were previously run sequentially despite being completely independent operations. Both were refactored to run in parallel using ThreadPoolExecutor with max\_workers=2.

| Before                                               | After                                             |
|------------------------------------------------------|---------------------------------------------------|
| dense_search() → wait → sparse_search() → wait → RRF | dense_search() ↗ (parallel) → both complete → RRF |

Estimated saving: ~0.3–0.5s per retrieval. Small but zero cost since both calls are already available.

### 3.2.3 Conditional Critique Skip

LangSmith traces showed that the agent's decision to call critique (after generate completes) takes ~25s due to the growing token context. Yet critique itself only takes ~1.2s. When retrieval quality is already strong — indicated by a high CrosEncoder rerank score — the critique step rarely triggers a retry and adds limited value.

A rerank score threshold was added: if the best rerank score across retrieved chunks exceeds 4.0, critique is skipped entirely and the generate output is returned directly. The system prompt was updated with this condition:

**Critique skip condition:** After generate(): check best\_rerank\_score from retrieve() result. If best\_rerank\_score  $\geq 4.0$  → SKIP critique() and return the draft directly. Otherwise call critique() as before.

| Config Key           | Value | Effect                                                                                    |
|----------------------|-------|-------------------------------------------------------------------------------------------|
| skip_critique_rerank | 4.0   | If best rerank score $\geq 4.0$ , skip critique — eliminates the 25s agent reasoning step |



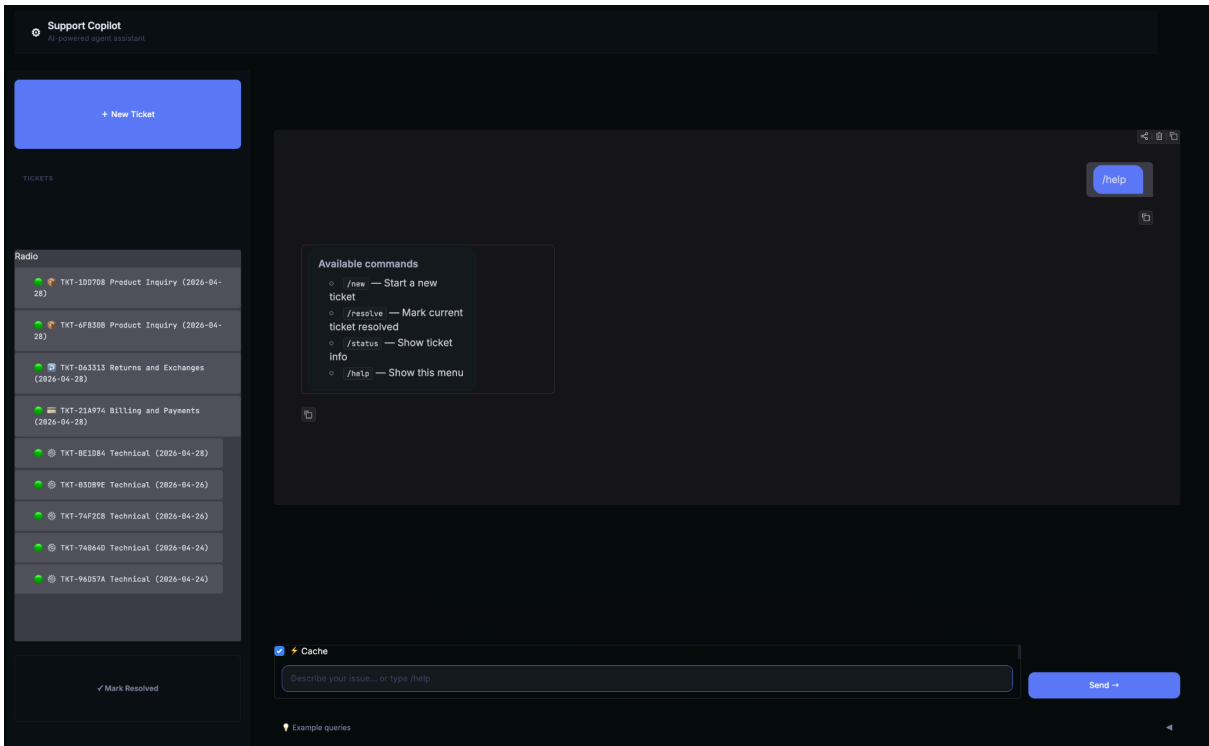
Estimated saving: ~26s on strong-retrieval queries. This is the largest single latency improvement available without replacing the ReAct agent.

### 3.2.4 Optimisation Summary

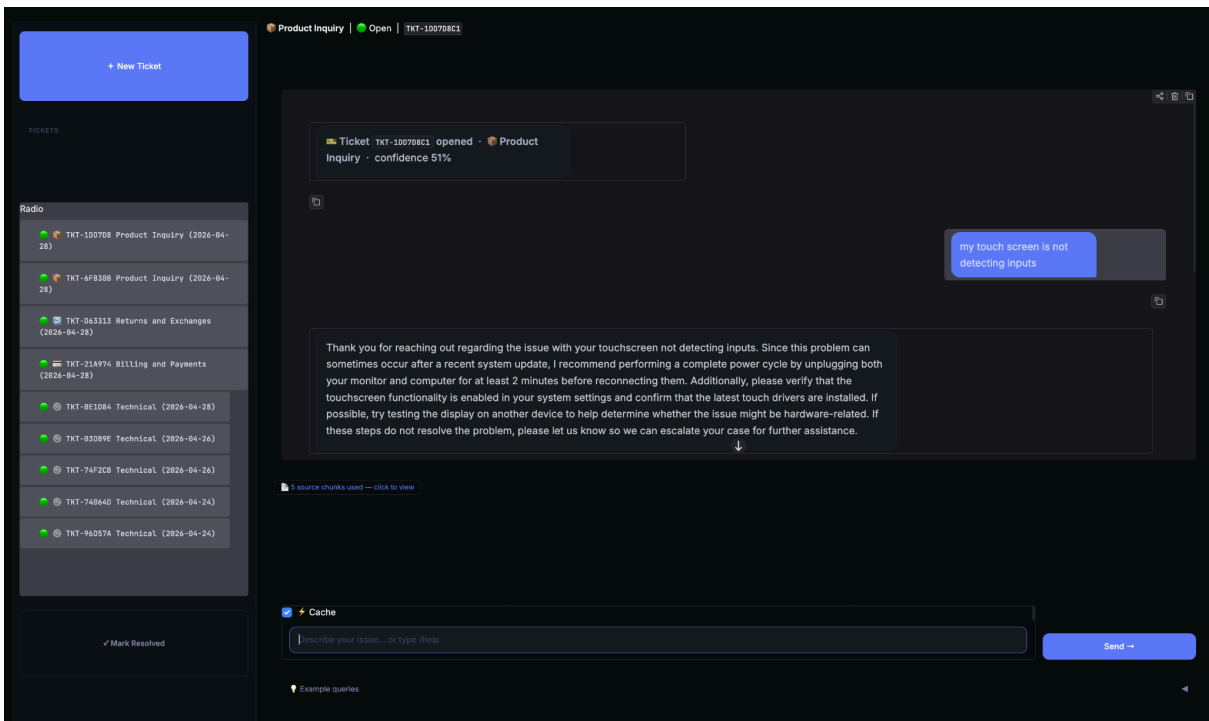
| Optimisation                                 | Files Changed                               | Expected Saving                | Risk                                                   |
|----------------------------------------------|---------------------------------------------|--------------------------------|--------------------------------------------------------|
| Skip query rewrite (confidence $\geq 0.75$ ) | retrieve.py, config.py                      | ~1.5s                          | Low — retrieval already strong at this confidence      |
| Parallel dense + sparse search               | retrieve.py                                 | ~0.3s                          | None — independent operations                          |
| Skip critique (rerank $\geq 4.0$ )           | orchestrator.py<br>system prompt, config.py | ~26s                           | Low — only skips when retrieval is demonstrably strong |
| Total estimated improvement                  | —                                           | ~28s per high-confidence query | —                                                      |

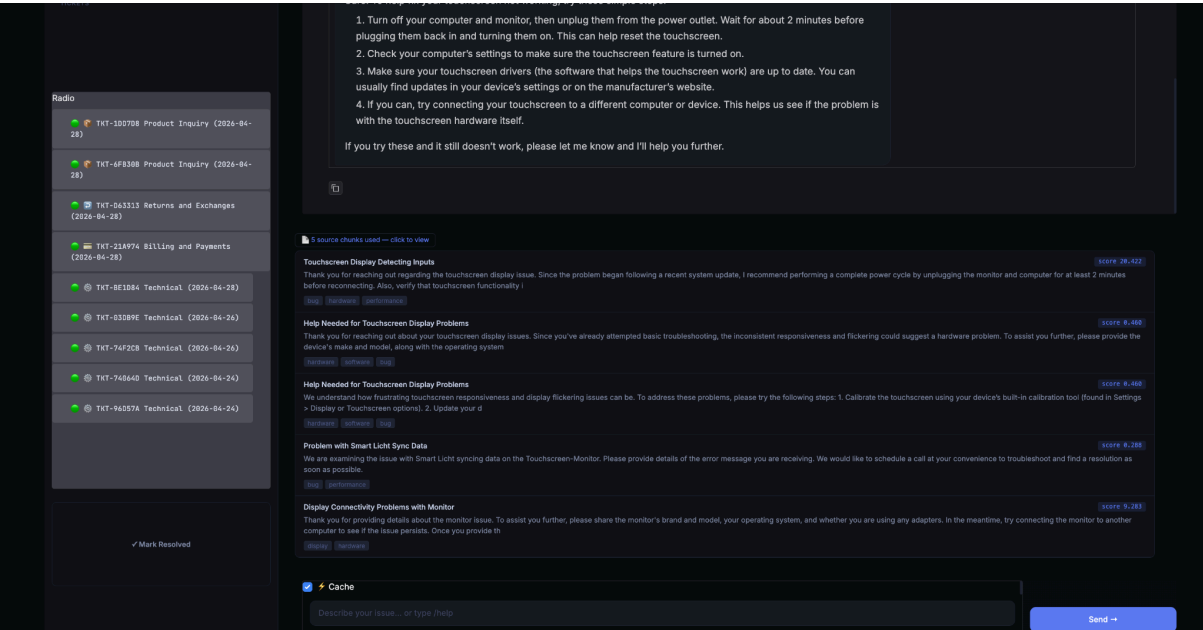
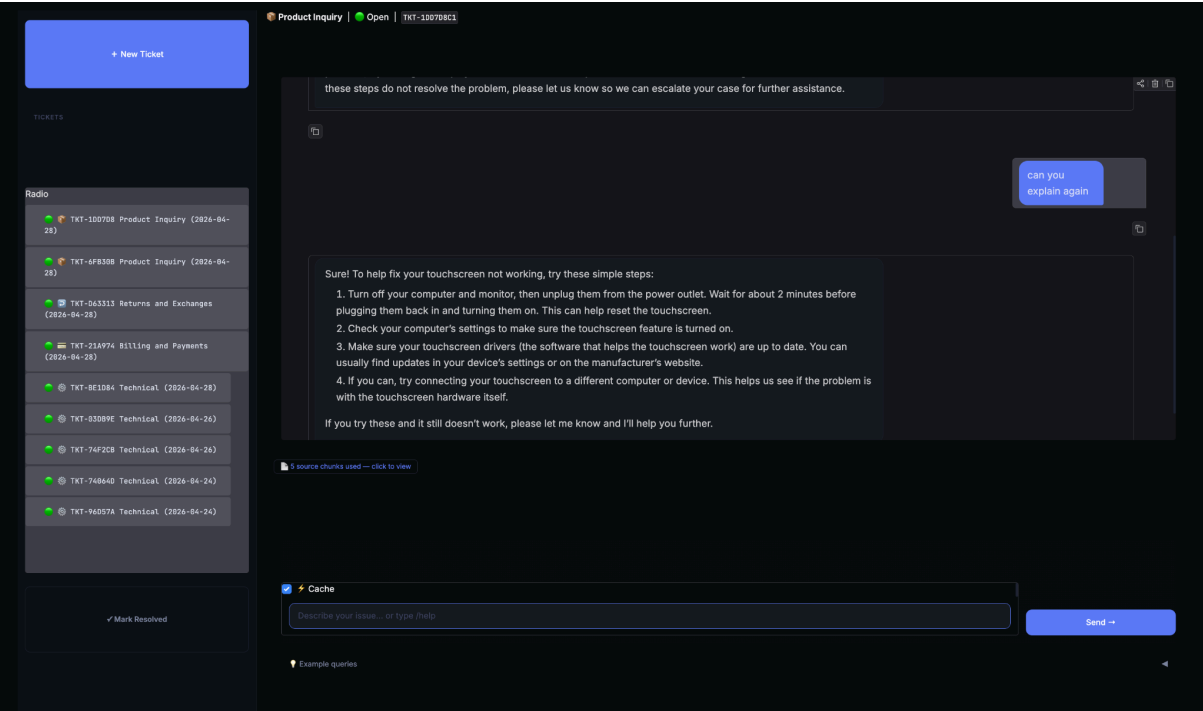
After these optimisations, a typical high-confidence query with strong retrieval should complete in approximately 40–45s instead of 70s. The remaining latency is dominated by the unavoidable sequential LLM calls in the ReAct agent reasoning loop.

# 5 Output Snips

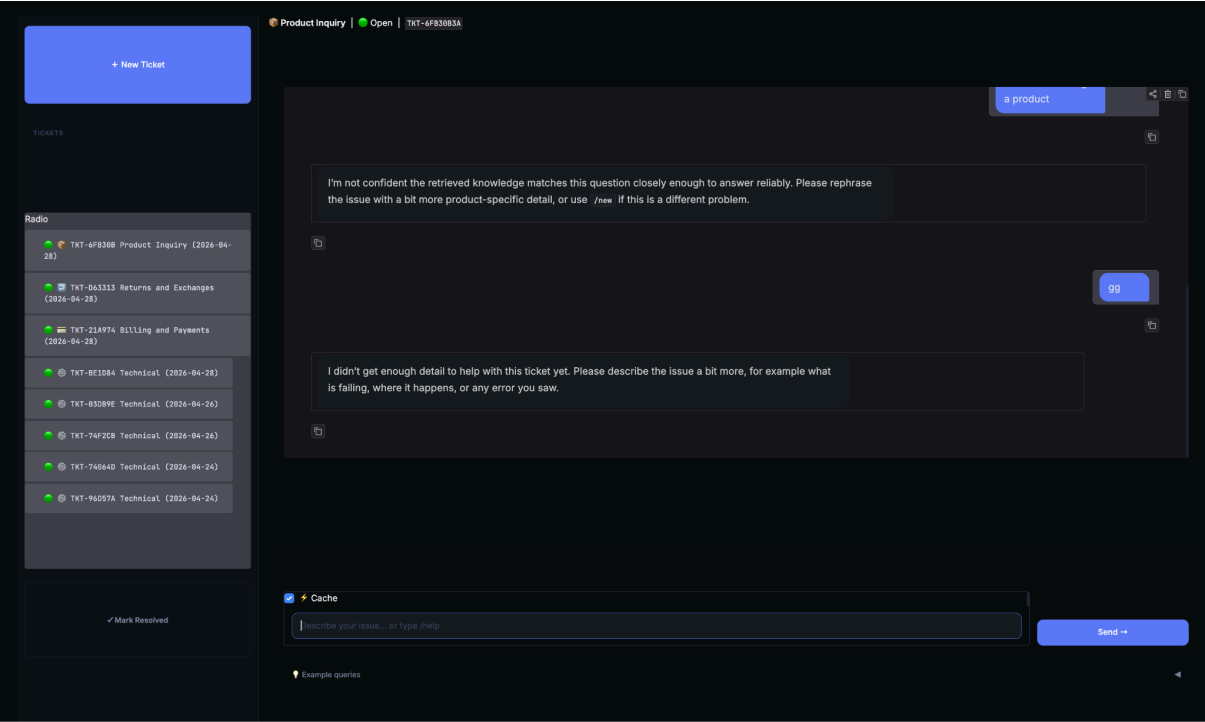
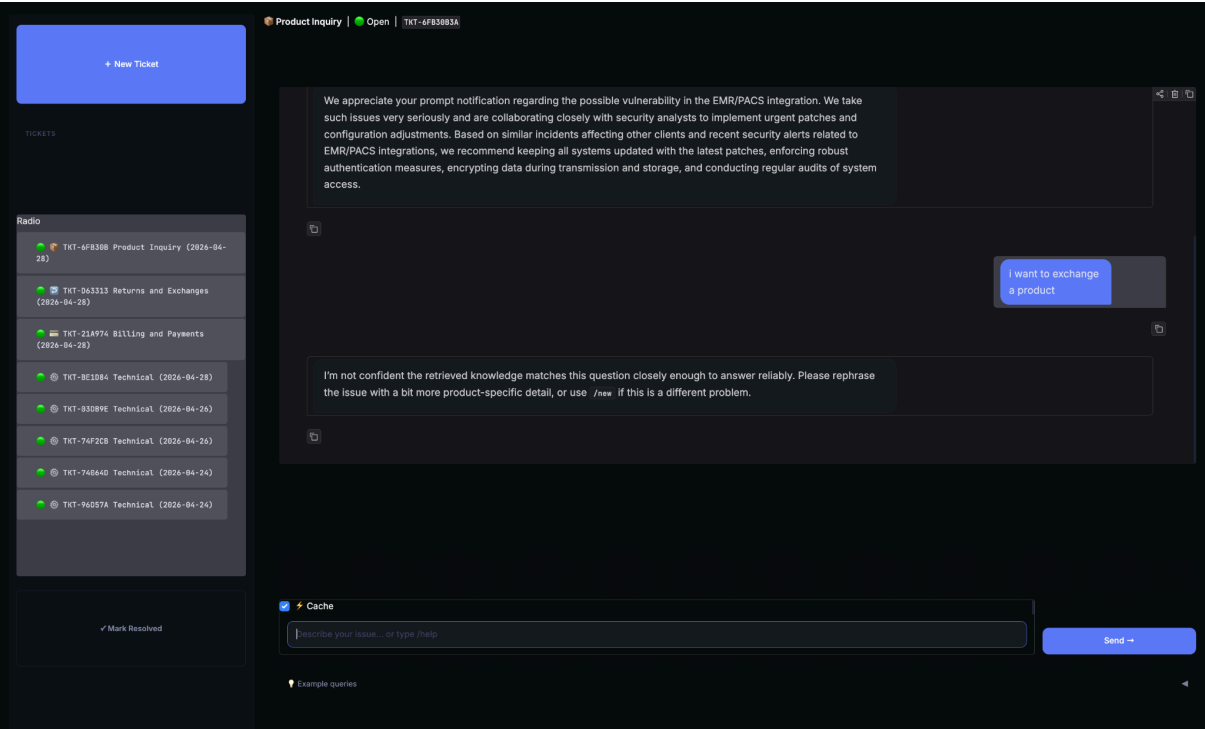


## Example 1: /cmds

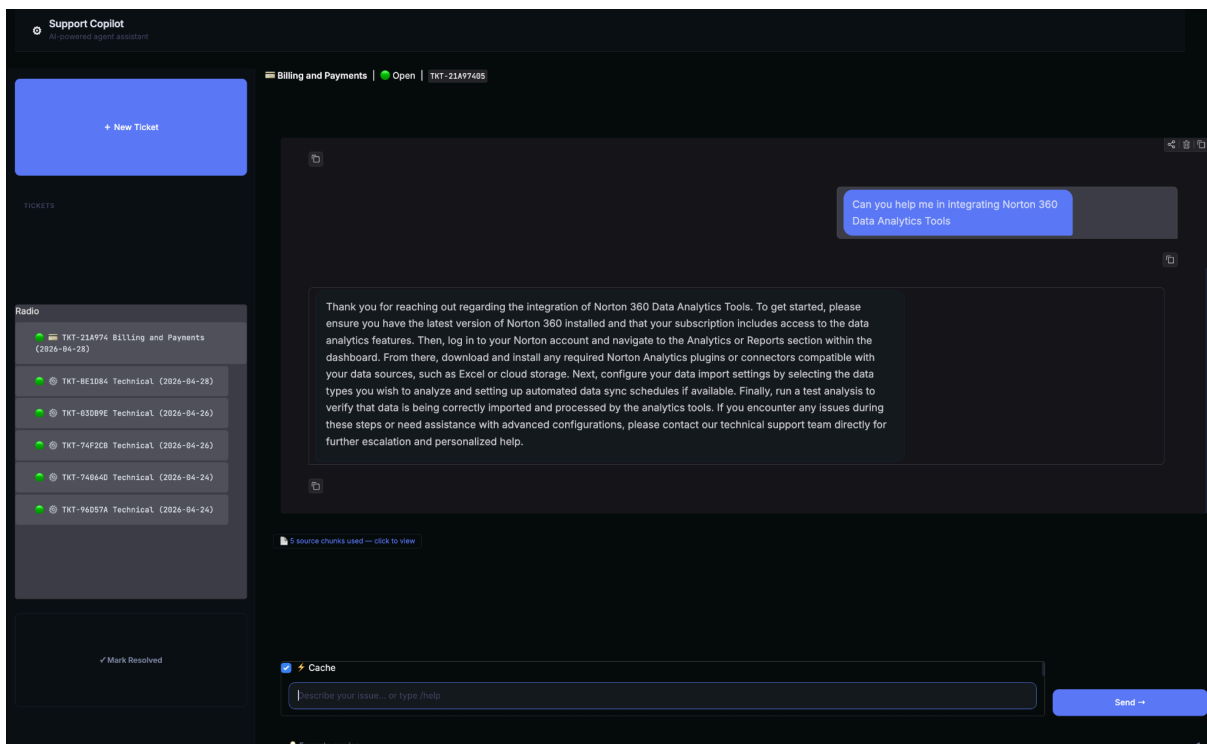
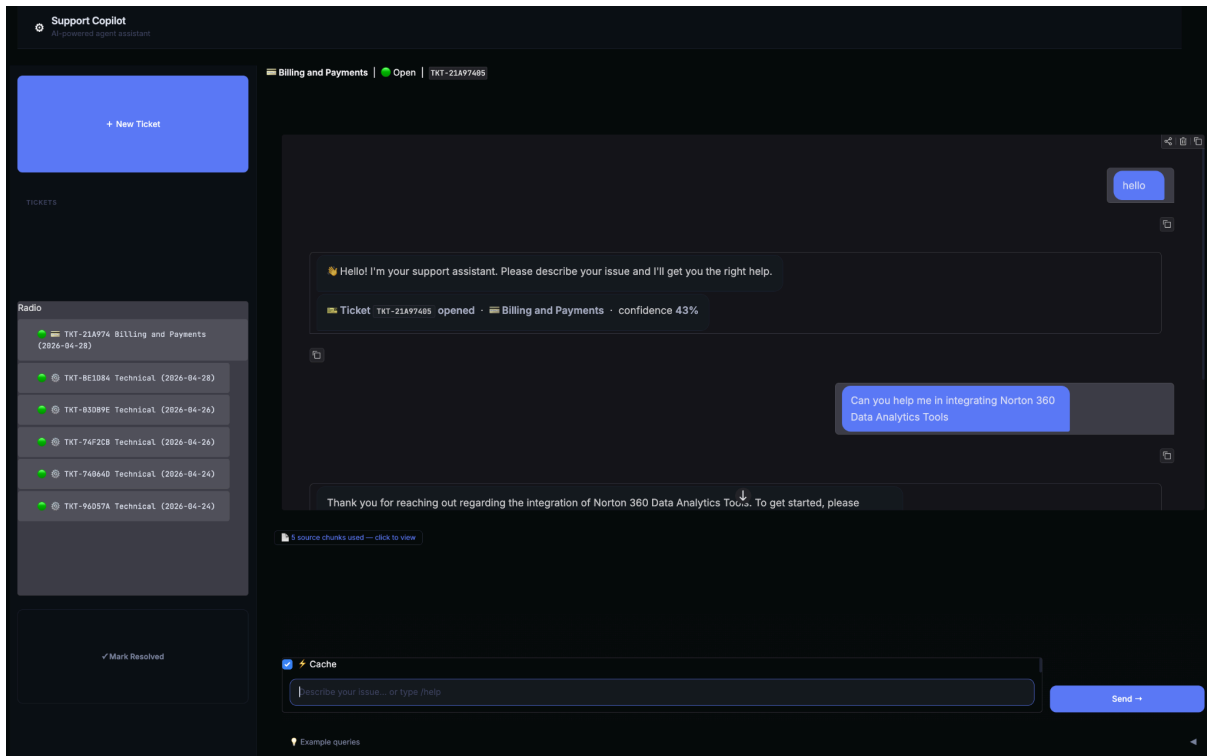




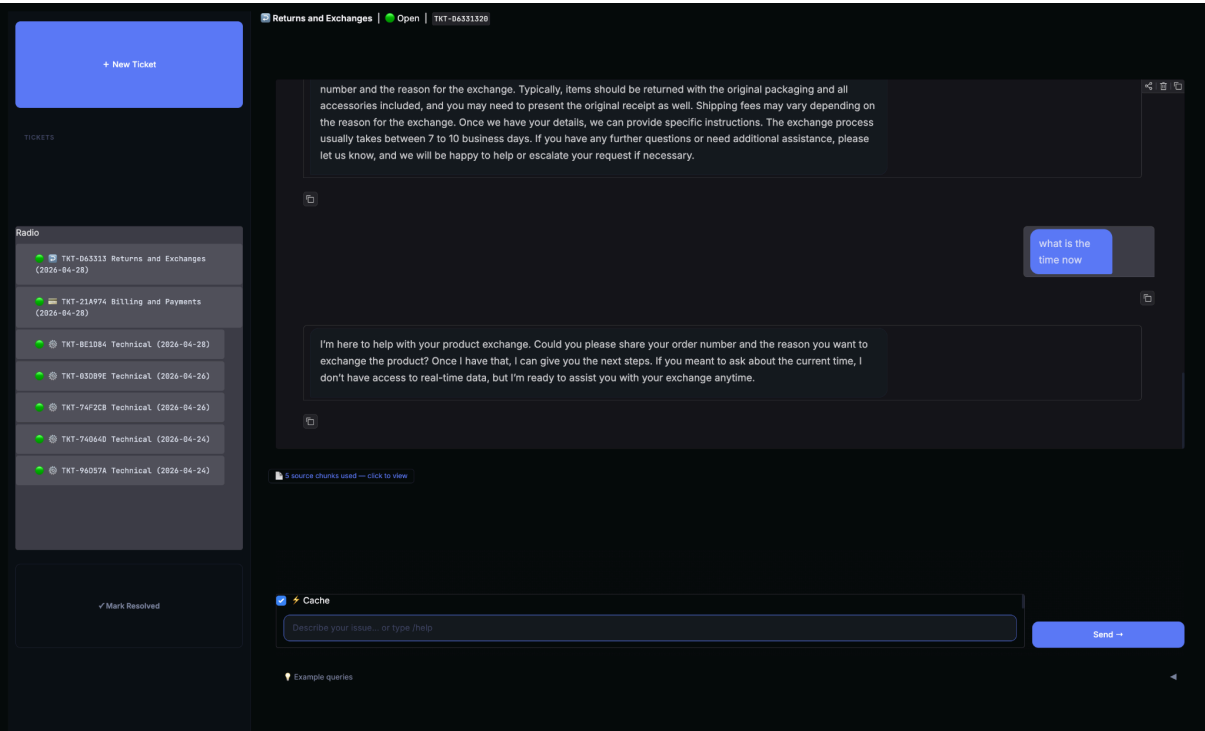
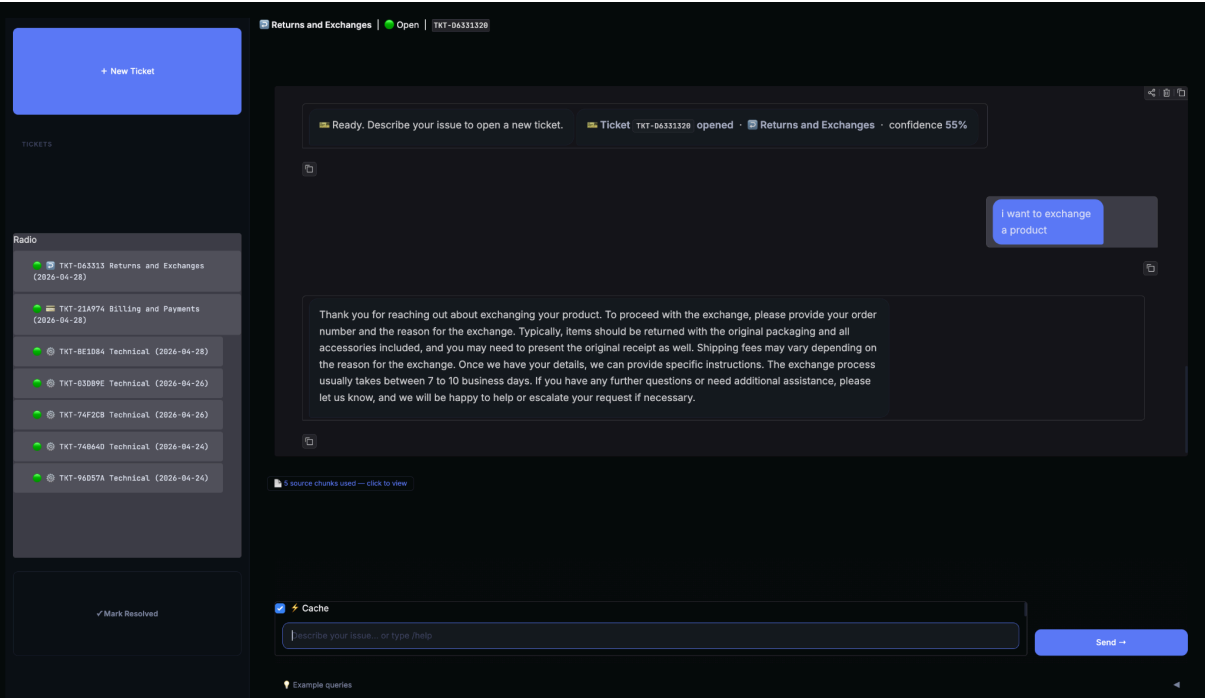
## Example 2: query, response, chunks, follow up



### Example 3: Topic drift, low information guard



## Example 4: Issues with dataset - misclassification.



## Example 5: Topic drift, with irrelevant question.

Support Copilot

AI-powered agent assistant

+ New Ticket

TICKETS

Radio

- TKT-BE1084 Technical (2024-04-28)
- TKT-03089E Technical (2024-04-24)
- TKT-74F2CB Technical (2024-04-24)
- TKT-74864D Technical (2024-04-24)
- TKT-96057A Technical (2024-04-24)

✓ Mark Resolved

Technical | Open | TKT-BE108488

Ticket TKT-BE108488 opened · Technical · confidence 76%

my data analytics tool crashes frequently

Dear Customer,

We understand that your data analytics tool crashes frequently, especially during peak usage times, and

5 source chunks used — click to view

Cache

Describe your issue... or type /help

Send →

Example queries

Support Copilot

AI-powered agent assistant

+ New Ticket

TICKETS

Radio

- TKT-BE1084 Technical (2024-04-28)
- TKT-03089E Technical (2024-04-24)
- TKT-74F2CB Technical (2024-04-24)
- TKT-74864D Technical (2024-04-24)
- TKT-96057A Technical (2024-04-24)

✓ Mark Resolved

Technical | Open | TKT-BE108488

Dear Customer,

We understand that your data analytics tool crashes frequently, especially during peak usage times, and we are here to help resolve this issue. To improve performance and reduce crashes, please try the following steps:

1. Clear the tool's cache and temporary files to free up memory.
2. Ensure your system meets the recommended hardware specifications, particularly CPU and RAM, as peak usage can strain resources.
3. Close any unnecessary applications running simultaneously to reduce system load during peak hours.
4. Update the data analytics tool to the latest version, as updates often include performance improvements and bug fixes.

If these steps do not resolve the issue, please let us know, and we will escalate your ticket to our technical engineering team for a deeper investigation. They may request logs and system diagnostics to

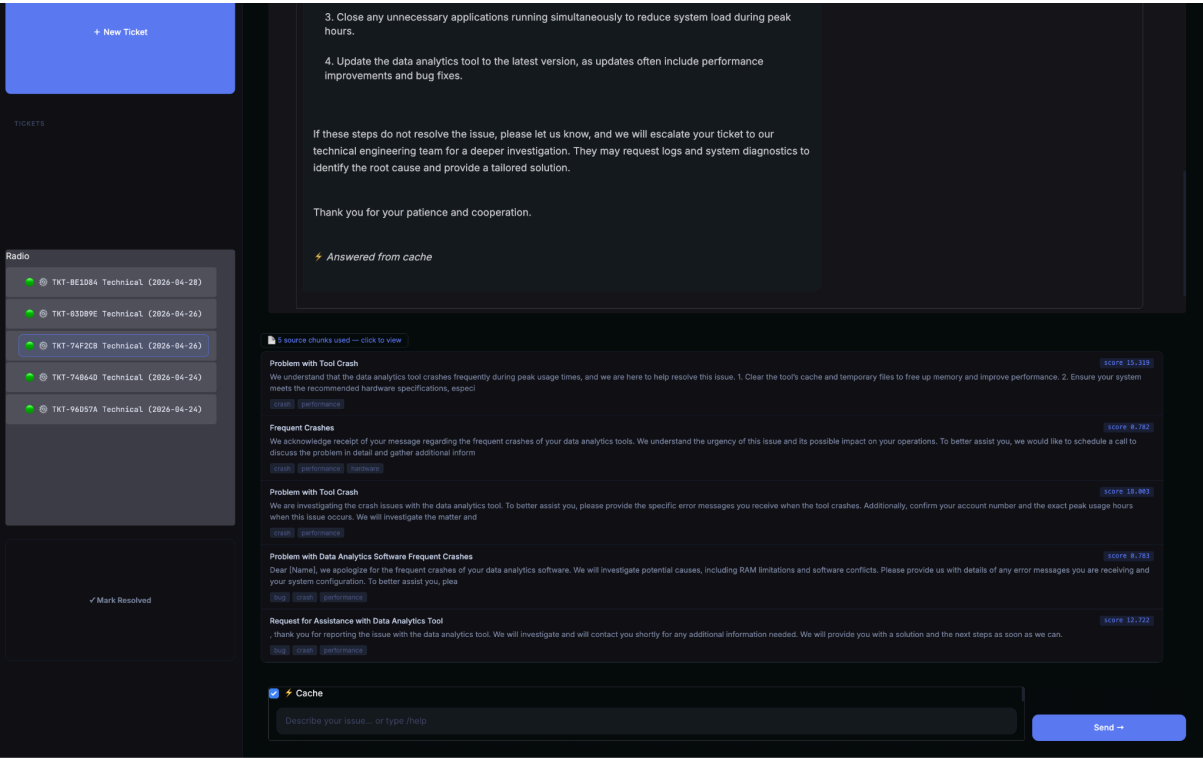
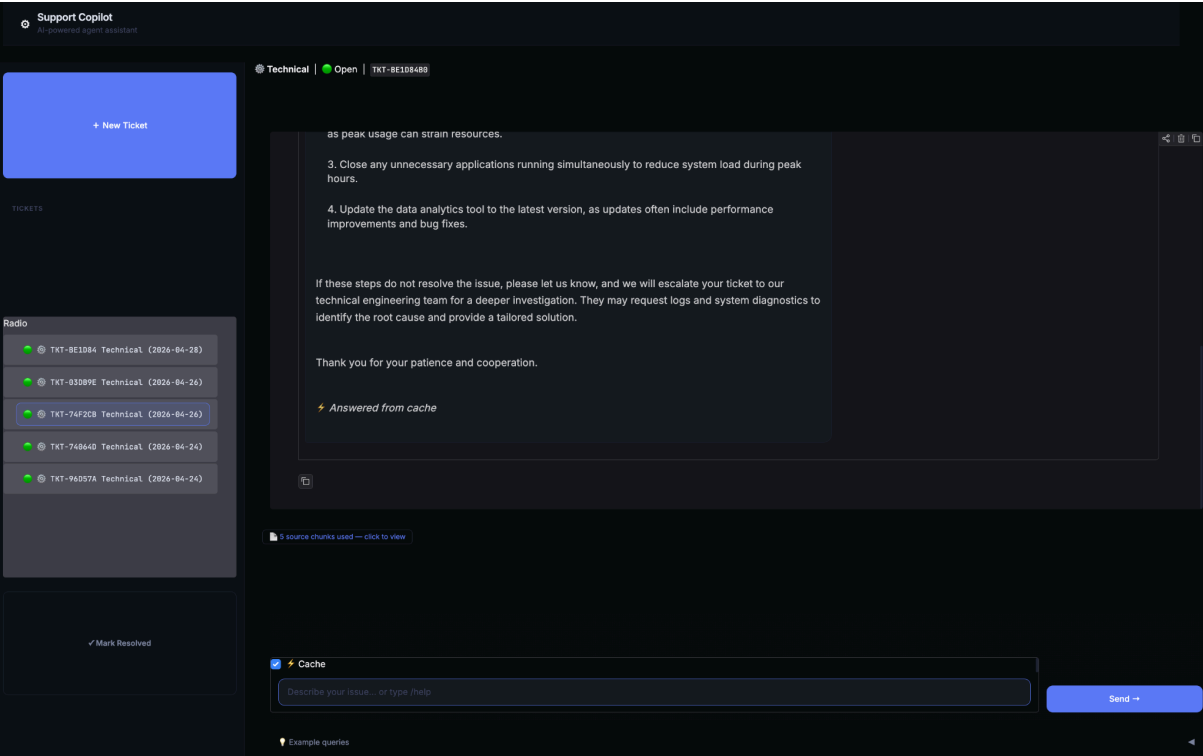
5 source chunks used — click to view

Cache

Describe your issue... or type /help

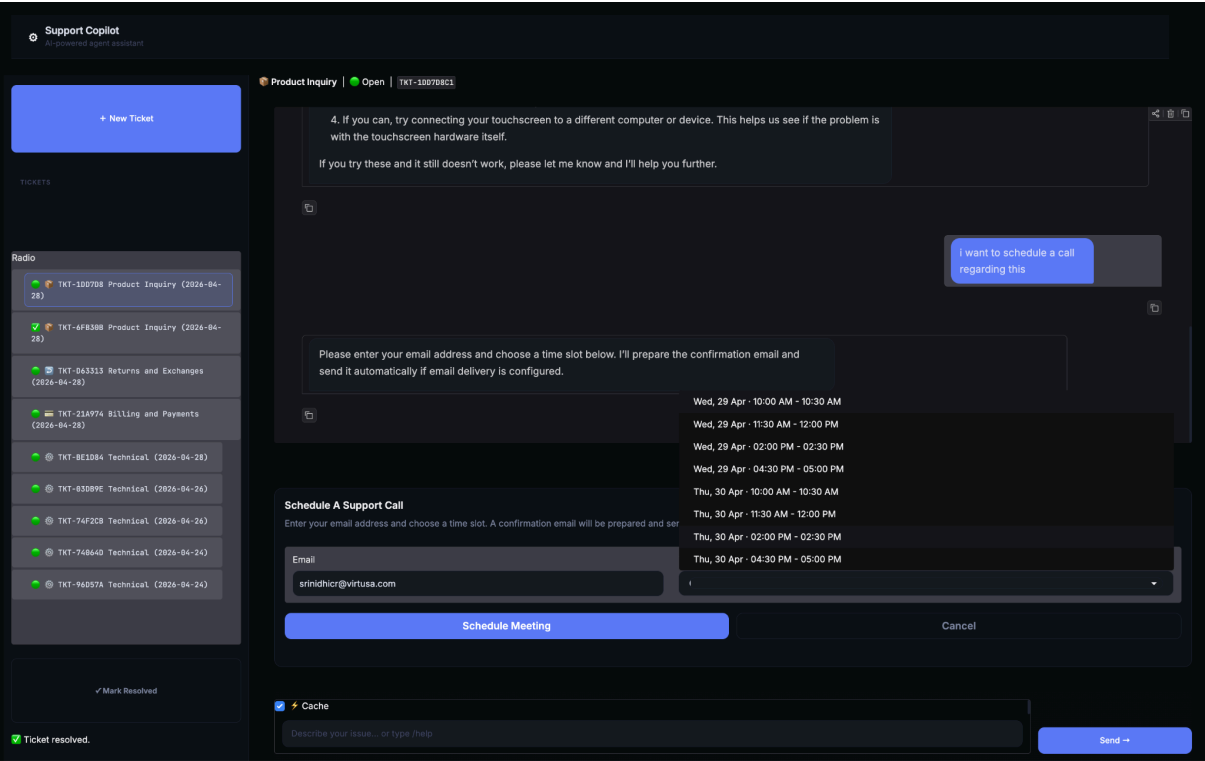
Send →

Example queries



## Example 5: Response from cache





## Example 6: Scheduling a call

## 6 Testing and Evaluation Coverage

| Script               | Coverage                                                                                                | Status      |
|----------------------|---------------------------------------------------------------------------------------------------------|-------------|
| test_chat_routing.py | 5 regression tests: short input / greeting / out-of-scope / same-issue rephrasing / schedule detection  | 5/5 passing |
| eval_main.py         | Dataset-backed retrieval: Hit@1/3/5 + MRR@5 for dense, sparse, hybrid across 6 categories (240 queries) | Complete    |
| test_pipeline.py     | End-to-end smoke test and batch test across 9 built-in queries including cache-pair test                | Complete    |
| test_retrieval.py    | Isolated retrieval test: keyword hit-rate across 5 query types (no LLM calls, zero API cost)            | Complete    |

### 6.1 Retrieval Evaluation Results (Full Breakdown)

| Category              | Method | Hit@1 | Hit@3 | Hit@5 | MRR@5 |
|-----------------------|--------|-------|-------|-------|-------|
| Billing and Payments  | Dense  | 85.0% | 95.0% | 97.5% | 0.902 |
| Billing and Payments  | Sparse | 100%  | 100%  | 100%  | 1.000 |
| Billing and Payments  | Hybrid | 100%  | 100%  | 100%  | 1.000 |
| General Inquiry       | Dense  | 85.0% | 87.5% | 97.5% | 0.887 |
| General Inquiry       | Sparse | 100%  | 100%  | 100%  | 1.000 |
| General Inquiry       | Hybrid | 97.5% | 97.5% | 100%  | 0.981 |
| Human Resources       | Dense  | 90.0% | 97.5% | 100%  | 0.938 |
| Human Resources       | Sparse | 100%  | 100%  | 100%  | 1.000 |
| Human Resources       | Hybrid | 92.5% | 100%  | 100%  | 0.963 |
| Product Inquiry       | Dense  | 82.5% | 92.5% | 92.5% | 0.871 |
| Product Inquiry       | Sparse | 100%  | 100%  | 100%  | 1.000 |
| Product Inquiry       | Hybrid | 95.0% | 100%  | 100%  | 0.975 |
| Returns and Exchanges | Dense  | 90.0% | 100%  | 100%  | 0.950 |
| Returns and Exchanges | Sparse | 100%  | 100%  | 100%  | 1.000 |
| Returns and Exchanges | Hybrid | 100%  | 100%  | 100%  | 1.000 |
| Technical             | Dense  | 85.0% | 97.5% | 97.5% | 0.904 |
| Technical             | Sparse | 100%  | 100%  | 100%  | 1.000 |
| Technical             | Hybrid | 100%  | 100%  | 100%  | 1.000 |
| Overall               | Dense  | 86.3% | 95.0% | 97.5% | 0.909 |
| Overall               | Sparse | 100%  | 100%  | 100%  | 1.000 |
| Overall               | Hybrid | 97.5% | 99.6% | 100%  | 0.986 |

